

BURT++: An Interactive AI-Powered Bug Reporting System for Mobile Applications

ANTU SAHA, William & Mary, USA

SAM BENNETT, William & Mary, USA

OSCAR CHAPARRO, William & Mary, USA

Mobile app developers rely on end-user bug reports to fix defects. However, these reports are often incomplete or ambiguous, making defects difficult to reproduce and resolve, increasing maintenance costs. We developed BURT++, an AI-powered interactive bug-reporting system that guides users through natural-language conversation. It identifies missing information, asks follow-up questions, and grounds responses in application information. By transforming informal descriptions into structured, developer-ready bug reports, BURT++ helps developers resolve defects more efficiently, improving software reliability. This enables users to benefit from more dependable software in their daily lives.

1 Problem Statement and Solution

Mobile applications have become an essential part of everyday life. People rely on apps for communication, banking, healthcare, education, transportation, productivity, and entertainment. Like all software systems, mobile apps contain bugs—problems that cause errors, unexpected behavior, or poor user experiences. When users encounter these issues, they often report them through bug-tracking systems such as GitHub Issues [2] and Bugzilla [1]. These bug reports serve as the primary communication channel between users and developers, helping developers understand what went wrong, reproduce the problem, and ultimately create a fix.

The quality of a bug report plays a critical role in the software maintenance process. High-quality reports help developers diagnose problems efficiently, reproduce defects on their devices, and identify the source code that requires modification. Unfortunately, many bug reports submitted by end users are incomplete, ambiguous, or missing critical information. Users may describe the problem they observed but omit important details such as the actions they performed, the screen where the issue occurred, or what they expected the application to do instead. Prior research has shown that poor-quality bug reports can lead to non-reproducible bugs [10], unresolved defects [22], longer bug-fixing times [13, 22], and defects that must be reopened after an attempted fix [22].

Despite decades of research and the widespread adoption of issue-tracking systems, this communication gap remains a persistent challenge. Most existing reporting tools rely on static forms that place the burden on users to know what information developers need. However, many users are not software experts and often struggle to communicate technical problems effectively. As a result, developers spend significant time requesting additional information before they can begin diagnosing and fixing a defect.

To address this challenge, we developed BURT++ (Interactive **B**ug **R**epor**T**ing for Mobile Apps), an AI-powered conversational bug reporting system. Rather than asking users to complete a rigid form, BURT++ engages them in a natural language conversation. The system analyzes the information provided by the user, identifies missing or unclear details, and asks targeted follow-up questions designed to gather the information developers need. The experience resembles interacting with an experienced software tester rather than filling out a questionnaire.

Authors' Contact Information: Antu Saha, William & Mary, Williamsburg, Virginia, USA, asaha02@wm.edu; Sam Bennett, William & Mary, Williamsburg, Virginia, USA, srbennett01@wm.edu; Oscar Chaparro, William & Mary, Williamsburg, Virginia, USA, oscarch@wm.edu.

What makes BURT++ particularly effective is its ability to reason about the application’s user interface while conducting the conversation. The system connects user descriptions to specific application screens, user actions, and navigation paths, allowing it to detect ambiguities and generate more precise clarification questions. Throughout the interaction, BURT++ continuously tracks the information it has collected and dynamically adapts its questions to focus only on unresolved details.

By helping users communicate software problems more effectively, BURT++ produces bug reports that are more complete, reproducible, and actionable. This reduces the effort developers spend gathering information, accelerates debugging and software maintenance, and ultimately improves the quality and reliability of the software that millions of people use every day.

2 Key Novelty of BURT++

The novelty of BURT++ lies in its integration of conversational artificial intelligence, application information reasoning, and dynamic information management into a single bug-reporting system. While existing tools improve individual aspects of bug reporting, BURT++ combines all three capabilities to help users create high-quality bug reports through an intelligent conversation.

Traditional bug-reporting platforms such as GitHub Issues [2] and Bugzilla [1] rely on static forms. Users must determine what information developers need and provide it themselves, often resulting in incomplete or ambiguous reports. Existing tools collect information but do little to help users improve report quality.

Prior research introduced interactive bug reporting through follow-up questions. For example, BURT [21] showed that conversational reporting can help users provide more complete information than traditional forms. However, these systems largely relied on predefined rules and lacked an understanding of the application being reported, limiting their ability to identify ambiguities or adapt to different apps.

BURT++ advances beyond these approaches in three key ways.

- First, BURT++ leverages large language models (LLMs) to conduct natural, human-like conversations. Rather than requiring users to complete a rigid template, it dynamically adapts its questions based on the information already provided, creating a more intuitive reporting experience.
- Second, BURT++ incorporates application-specific knowledge into the reporting process. It reasons about application screens, user interface elements, interactions, and navigation paths while interpreting a user’s description. This allows the system to detect missing or unclear information and generate clarification questions tailored to the application’s actual functionality. This capability builds on recent findings showing that application-specific GUI information can significantly improve bug report quality [19].
- Third, BURT++ maintains an internal representation of the bug being reported. It tracks critical information such as the observed problem, expected behavior, triggering actions, affected screens, and steps to reproduce (S2Rs) [19]. By identifying what information is missing or uncertain, the system asks only the questions needed to complete the report, making the process more efficient and personalized.

Together, these innovations enable BURT++ to generate bug reports that are more complete, reproducible, and actionable than those produced by conventional reporting methods, representing a new approach to communication between software users and developers.

BURT++ : An Interactive AI-Powered Bug Reporting System for Mobile Applications



Fig. 1. BURT++'s User Interface with an Example User Conversation and Generated Report

3 How BURT++ Works

BURT++ transforms an incomplete user description into a structured, developer-ready bug report through an interactive conversation (see Figure 1). The system was designed to bridge the communication gap between non-technical users and software developers by actively guiding users toward providing the information needed to diagnose and reproduce software defects [4].

The reporting process begins when a user describes a problem in natural language. Rather than immediately generating a bug report, BURT++ analyzes the user's description and extracts information related to five critical bug-reporting elements identified by prior software engineering research: the observed buggy behavior, the expected behavior, the triggering user interaction, the triggering application screen, and the steps required to reproduce the bug [19].

After extracting this information, BURT++ maintains an internal representation of the bug, referred to as a bug state. Each element in the bug state is assigned a confidence level that indicates whether the information is unknown, ambiguous, inferred, or confirmed. This enables the system to continuously assess the completeness and quality of the report as the conversation progresses.

A key innovation of BURT++ is its ability to reason about the application's user interface while interacting with the user. The system uses an application execution model that contains information about application screens, interface elements, user interactions, and navigation paths. User descriptions are mapped onto this model so that BURT++ can

understand which parts of the application are being discussed. This application information reasoning allows the system to identify ambiguities that would be difficult to detect using the user description alone.

When critical information is missing or unclear, BURT++ generates targeted follow-up questions. For example, if a user reports that an application "crashes when I save a file," the system may ask which screen the user was on, what action triggered the problem, or what behavior was expected instead. Unlike traditional reporting forms that ask every user the same questions, BURT++ dynamically adapts its questions based on the information already collected. Once an element has been confirmed, the system focuses only on unresolved uncertainties.

The conversation continues until BURT++ determines that sufficient information has been collected. The system then synthesizes the information into a structured bug report that developers can use to reproduce and diagnose the defect. The structured bug report includes four sections: title, observed behavior, expected behavior, and steps-to-reproduce (S2Rs).

This approach combines conversational AI, application information reasoning, and dynamic bug-state management into a single workflow. By continuously evaluating report completeness and asking targeted clarification questions, BURT++ helps users provide richer information with less effort, resulting in bug reports that are more complete, reproducible, and actionable for software developers.

4 Potential Benefits

BURT++ has the potential to generate significant economic, societal, and environmental benefits by improving the way software defects are communicated between users and developers.

4.1 Economic Benefits

Software companies invest substantial resources in software maintenance, debugging, and quality assurance. Studies have estimated that software maintenance can account for more than half of the total cost of a software system over its lifetime, with debugging and defect resolution representing a significant portion of that effort [11]. When bug reports are incomplete or ambiguous, developers often spend considerable time requesting additional information, attempting to reproduce defects, and determining what actually happened before they can begin fixing the problem.

BURT++ addresses this challenge by guiding users toward creating more complete and reproducible bug reports. By providing developers with higher-quality information from the start, the system can reduce time spent gathering clarification, accelerate defect resolution, and improve engineering productivity. These improvements can translate into meaningful cost savings for organizations that manage large software systems and receive thousands of user reports each year.

The invention also presents commercialization opportunities. BURT++ can be integrated into existing issue-tracking systems, customer support platforms, and software development workflows through Software-as-a-Service (SaaS) offerings and integrations with widely used development platforms.

4.2 Societal Benefits

Modern society depends heavily on software systems for communication, healthcare, education, transportation, commerce, and public services. The quality and reliability of these systems directly affect millions of people.

BURT++ improves communication between software users and software developers by helping users express software problems more effectively, regardless of their technical expertise. Most software users are not programmers and often struggle to describe technical issues in a way that developers can understand. Prior research has shown that low-quality

bug reports can hinder defect resolution and increase the effort required by developers to diagnose problems [5]. By lowering this communication barrier, BURT++ enables a broader population of users to contribute meaningful feedback that can improve software quality.

Faster identification and resolution of software defects can improve the reliability, usability, and safety of software systems. Users benefit from quicker bug fixes, while development teams gain a clearer understanding of real-world problems experienced by their customers. In this way, BURT++ helps create a more effective feedback loop between software developers and users.

Overall, BURT++ has the potential to improve software maintenance efficiency, reduce development costs, strengthen communication between users and developers, and contribute to more reliable software systems that benefit organizations and society as a whole.

5 Student Contributions and Initiative

BURT++ was conceived and developed by the student team consisting of Antu Saha and Sam Bennett under the supervision of Dr. Oscar Chaparro at William & Mary. Professor Chaparro provided research mentorship, technical guidance, and feedback throughout the project, while the student inventors were responsible for conceiving, designing, and developing the invention.

Antu Saha led the development of the research vision and technical direction behind BURT++. He identified key shortcomings in existing bug-reporting approaches, formulated the fundamental ideas driving the invention, designed the overall system architecture, and developed many of the AI-driven techniques that enable conversational bug reporting, application-information reasoning, and adaptive clarification. He also led the project from its initial conception through the design, implementation, and validation of a working prototype. In addition, he coordinated the project's research agenda and played a central role in transforming the invention from a research idea into a practical software system.

Sam Bennett played a key role in translating the invention into a robust and integrated software platform. He made substantial contributions to the implementation of the system, including frontend and backend development, software engineering, agent orchestration, infrastructure design, and integration of the various components that power BURT++. His contributions were critical to operationalizing the invention and ensuring that the platform could be deployed, evaluated, and used in realistic software development environments.

Dr. Oscar Chaparro served as the faculty advisor and professional reference for the project. His role focused on providing research mentorship, technical feedback, and guidance throughout the development process. He helped evaluate research directions, reviewed design decisions, provided feedback on prototypes, and supported the students as they refined the invention.

The success of BURT++ reflects the complementary contributions of both student inventors. Antu's leadership in research and innovation, combined with Sam's expertise in system engineering and implementation, enabled the creation of a novel AI-powered bug-reporting system with the potential for significant impact on software development and maintenance.

6 Future Improvements and Broader Applicability

While BURT++ currently focuses on mobile applications, the underlying ideas behind the invention are not limited to mobile software. The core capabilities of conversational bug elicitation, application information reasoning, adaptive clarification, and structured report generation can be applied to other types of software systems, including web applications, desktop applications, enterprise software, and software-as-a-service (SaaS) platforms.

Extending BURT++ to new application domains is feasible because the system architecture was designed to separate the bug-reporting workflow from the application-specific information used during reasoning. The primary effort required for adaptation involves constructing an application model that captures the relevant screens, user interactions, and navigation structure of the target system. Once this information is available, the existing conversational and reasoning components can be reused with minimal modification.

In future work, we plan to expand BURT++ beyond mobile applications and investigate its use in a broader range of software ecosystems. We also plan to incorporate additional sources of information, such as screenshots, screen recordings, and execution traces, to further improve the quality and completeness of generated bug reports. These extensions would increase the applicability of the invention and enable a wider range of organizations to benefit from AI-assisted bug reporting.

7 Intellectual Property and Commercialization Status

BURT++ is a research-stage invention developed at William & Mary to improve software maintenance through AI-assisted bug reporting. The system has been implemented as a functional prototype that supports conversational bug reporting, application information reasoning, adaptive clarification, and structured report generation. While formal intellectual property protection has not yet been pursued, the novelty and commercialization potential of the invention have been discussed within the research team.

The invention is attributed to the student inventors, Antu Saha and Sam Bennett, who conceived, designed, and developed BURT++ under the supervision of Dr. Oscar Chaparro. Any future publications, intellectual property filings, or technology transfer activities resulting from this work will appropriately recognize the contributions of the inventors and advisor in accordance with university policies.

Although BURT++ is currently a research prototype, it was designed with practical deployment in mind. The system generates structured, developer-ready bug reports that are compatible with existing software maintenance workflows and issue-tracking practices. Adapting the tool for integration with commercial bug-tracking platforms or embedding it within mobile and web applications would require additional engineering effort, but the underlying architecture was designed to support such extensions. These opportunities provide a potential pathway toward commercialization through SaaS offerings, software quality assurance tools, and industry partnerships. We plan to explore technology transfer and commercialization opportunities as the technology matures and undergoes further evaluation.

8 Literature Research

Our literature research identified two primary areas of prior work closely related to BURT++: (1) techniques for assessing and improving bug report quality and (2) interactive bug-reporting systems. This report summarizes the most relevant research in these areas and highlights the key differences between BURT++ and existing approaches.

8.1 Assessing and Improving Bug Report Quality

The most foundational work focuses on understanding and improving bug report quality. Empirical studies have shown that bug reports in traditional issue tracking systems are often incomplete or ambiguous, frequently missing key information such as reproduction steps, expected behavior, or execution context, which leads to non-reproducible bugs and delayed fixes [5, 10, 13, 22]. Past research has proposed methods to assess bug report quality from multiple angles, ranging from general characteristics such as readability, coherence, and completeness to the evaluation of specific components like S2Rs. Zimmermann [23] classified bug reports as bad, neutral, or good by analyzing features

including keyword completeness, presence of patches, screenshots, and readability. Dit *et al.* [9] assessed report quality by analyzing the coherence of comments in bug report discussions, while Linstead *et al.* [15] measured coherence using an information-theoretic approach based on entropy distributions of latent topics.

Chaparro *et al.* [7] proposed EULER to evaluate S2R quality via heuristics that map keywords in reports to app UI information. More recently, Mahmud *et al.* [16] introduced ASTROBR, which uses LLMs to assess S2R quality. Imran *et al.* [14] aimed to enhance incomplete reports by generating follow-up clarification questions to developers. Researchers have also proposed techniques to improve the entire bug report quality by leveraging LLMs [3, 6, 8, 19].

8.2 Interactive Bug Reporting

Researchers have proposed systems for interactive bug reporting that also aim to improve the quality of bug reports. Moran *et al.* proposed FUSION [17] that allows reporters to choose available actions and GUI components from dropdown lists, resulting in more structured and comprehensive bug reports. Fazzini *et al.* proposed EBUG [12] that extends FUSION and suggests potential S2Rs to the reporters alongside the dropdown lists available to FUSION. Song *et al.* proposed BURT [20, 21], a chatbot that guides the reporters and verifies the quality of bug information in real-time, providing suggestions to the reporters. Pourasad *et al.* [18] proposed FEEDALDE, which analyzes application information to ask follow-up questions to enhance the feedback reports of mobile applications.

8.3 How BURT++ is Different?

BURT++ introduces a new paradigm for bug reporting that combines conversational AI, application information reasoning, and confidence-driven bug-state management to actively collaborate with users during report creation.

Prior interactive bug reporting systems such as FUSION [17], EBUG [12], and BURT [20, 21] guide users while they report defects, helping them provide more structured information. However, these systems primarily rely on predefined workflows, templates, or rule-based interactions and have a limited ability to reason about the application’s functionality while interpreting user descriptions. While FEEDALDE [18], asks follow-up questions to improve the feedback report, it does not generate structured bug reports with various bug report elements (*e.g.*, observed behavior, steps-to-reproduce, *etc.*). Additionally, recent LLM-based bug report improvement approaches [3, 6, 8, 19] focus on assessing or improving bug reports after they have been submitted, generating higher-quality reports from user-provided information but remaining largely disconnected from the reporting process itself.

BURT++ bridges this gap through a unified framework that actively collaborates with users while a bug report is being created. The system combines conversational AI, application information reasoning, and confidence-driven bug-state management to continuously assess the quality of the information being collected. To the best of our knowledge, none of the aforementioned works integrates these three features in a unified framework.

As users describe a defect, BURT++ not only extracts bug-related information from the conversation but also grounds that information in application-specific metadata, including screens, user interface components, interactions, and navigation paths. This allows the system to detect ambiguities, identify missing information, and generate clarification questions tailored to the actual functionality of the application.

In addition, BURT++ maintains a dynamic representation of the reported bug that tracks the completeness and confidence of critical reporting elements throughout the conversation. Rather than asking generic follow-up questions or relying on predefined reporting flows, the system adapts its behavior based on the current state of the report and focuses only on unresolved uncertainties. By integrating conversational elicitation, application-specific reasoning, and adaptive clarification into a single reporting workflow, BURT++ transforms bug reporting from a passive information

collection task into an intelligent collaborative process, representing a significant advancement over both existing interactive bug reporting systems and recent AI-assisted bug report improvement techniques.

References

- [1] 2026. BugZilla Issue Tracker - <https://bugzilla.mozilla.org>.
- [2] 2026. GitHub Issue Tracker - <https://github.com/features/issues>.
- [3] Jagrit Acharya and Gouri Ginde. 2025. Can We Enhance Bug Report Quality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation. *arXiv preprint arXiv:2504.18804* (2025).
- [4] Jorge Aranda and Gina Venolia. 2009. The secret life of bugs: Going past the errors and omissions in software repositories. In *2009 IEEE 31st international conference on software engineering*. IEEE, 298–308.
- [5] Nicolas Bettenburg, Sascha Just, Adrian Schröter, Cathrin Weiss, Rahul Premraj, and Thomas Zimmermann. 2008. What Makes a Good Bug Report?. In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Atlanta, Georgia). 308–318.
- [6] Lili Bo, Wangjie Ji, Xiaobing Sun, Ting Zhang, Xiaoxue Wu, and Ying Wei. 2024. ChatBR: Automated assessment and improvement of bug report quality using ChatGPT. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA). 1472–1483.
- [7] Oscar Chaparro, Carlos Bernal-Cárdenas, Jing Lu, Kevin Moran, Andrian Marcus, Massimiliano Di Penta, Denys Poshyvanyk, and Vincent Ng. 2019. Assessing the Quality of the Steps to Reproduce in Bug Reports. In *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia). 86–96.
- [8] Seojin Choi and Geunseok Yang. 2025. AgentReport: A Multi-Agent LLM Approach for Automated and Reproducible Bug Report Generation. *Applied Sciences* 15, 22 (2025), 11931.
- [9] Bogdan Dit and Andrian Marcus. [n. d.]. Improving the readability of defect reports. In *Proceedings of the 2008 International Workshop on Recommendation Systems for Software Engineering (RSSE)*. 47–49.
- [10] Mona Erfani Joorabchi, Mehdi Mirzaaghaei, and Ali Mesbah. 2014. Works for me! characterizing non-reproducible bug reports. In *Proceedings of the 11th working conference on mining software repositories*. 62–71.
- [11] Len Erlikh. 2000. Leveraging legacy system dollars for e-business. *IT professional* 2, 3 (2000), 17–23.
- [12] Mattia Fazzini, Kevin Moran, Carlos Bernal-Cárdenas, Tyler Wendland, Alessandro Orso, and Denys Poshyvanyk. 2023. Enhancing Mobile App Bug Reporting via Real-Time Understanding of Reproduction Steps. *IEEE Transactions on Software Engineering* 49, 3 (March 2023), 1246–1272.
- [13] Philip J. Guo, Thomas Zimmermann, Nachiappan Nagappan, and Brendan Murphy. 2010. Characterizing and predicting which bugs get fixed: an empirical study of Microsoft Windows. In *Proceedings of the 32nd International Conference on Software Engineering*. 495–504.
- [14] Mia Mohammad Imran, Agnieszka Ciborowska, and Kostadin Damevski. 2021. Automatically Selecting Follow-up Questions for Deficient Bug Reports. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 167–178.
- [15] Erik Linstead and Pierre Baldi. 2009. Mining the coherence of GNOME bug reports with statistical topic models. In *Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories*. 99–102. doi:10.1109/MSR.2009.5069486
- [16] Junayed Mahmud, Antu Saha, Oscar Chaparro, Kevin Moran, and Andrian Marcus. 2025. Combining language and app ui analysis for the automated assessment of bug reproduction steps. *arXiv preprint arXiv:2502.04251* (2025).
- [17] Kevin Moran, Mario Linares-Vásquez, Carlos Bernal-Cárdenas, and Denys Poshyvanyk. 2015. Auto-completing Bug Reports for Android Applications. In *Proceedings of the Joint Meeting on Foundations of Software Engineering (FSE’15)*. 673–686.
- [18] Ali Ebrahimi Pourasad, Meyssam Saghir, and Walid Maalej. 2026. FeedAlde: Guiding App Users to Submit Rich Feedback Reports by Asking Context-Aware Follow-Up Questions. *arXiv preprint arXiv:2603.04244* (2026).
- [19] Antu Saha, Atish Kumar Dipongkor, Sam Bennett, Kevin Moran, Andrian Marcus, and Oscar Chaparro. 2026. Automated Generation of High-Quality Bug Reports for Android Applications. *arXiv preprint arXiv:2604.01148* (2026).
- [20] Yang Song, Junayed Mahmud, Nadeeshan De Silva, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2023. BURT: A Chatbot for Interactive Bug Reporting. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE’23)*. 170–174.
- [21] Yang Song, Junayed Mahmud, Ying Zhou, Oscar Chaparro, Kevin Moran, Andrian Marcus, and Denys Poshyvanyk. 2022. Toward interactive bug reporting for (android app) end-users. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 344–356.
- [22] Thomas Zimmermann, Nachiappan Nagappan, Philip J. Guo, and Brendan Murphy. 2012. Characterizing and predicting which bugs get reopened. In *Proceedings of the International Conference on Software Engineering (ICSE’12)*. 1074–1083.
- [23] Thomas Zimmermann, Rahul Premraj, Nicolas Bettenburg, Sascha Just, Adrian Schroter, and Cathrin Weiss. 2010. What Makes a Good Bug Report? *IEEE Transactions on Software Engineering* 36, 5 (2010), 618–643. doi:10.1109/TSE.2010.63